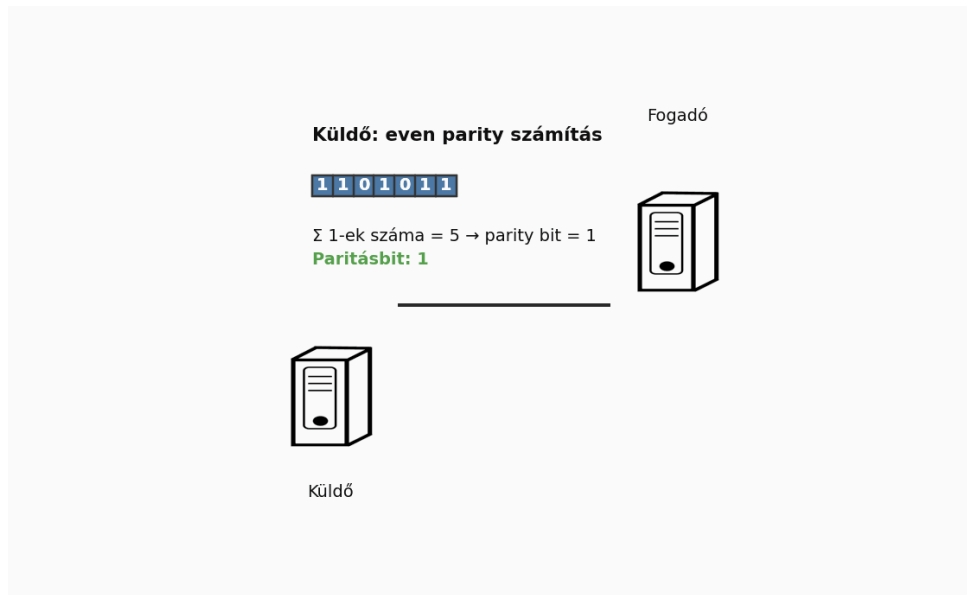




Alapfogalmak

- **Kódszó (c):** Fix hosszúságú bináris sorozat, amely az információ egy részét hordozza.
- **Kódkönyv (S):** A kód által használt érvényes kódszavak halmaza.
- **Kódszóhossz (n):** Minden kódszó bitjeinek száma.
- **Hamming-távolság ($d(x, y)$):** Két kódszó különböző pozícióinak száma.
- **Minimális távolság ($d(S)$):** A kódkönyvben bármely két különböző kódszó közötti legkisebb Hamming-távolság.

Hibafelismerő és hibajavító képesség

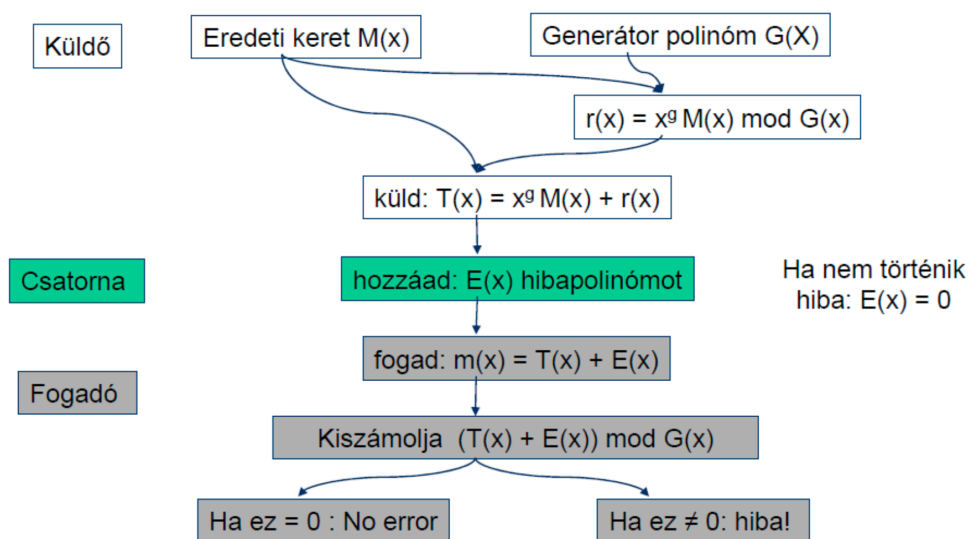
- **Hibafelismerés:** Ha a minimális távolság d , akkor a kód képes $d - 1$ hibát felismerni.
- **Hibajavítás:** A kód képes $\lfloor \frac{d-1}{2} \rfloor$ hibát javítani.
- **Példa:** Ha $d = 3$, akkor a kód felismer max. 2 hibát, és javítani tud 1 hibát.

Példa kódkönyv elemzése

Adott kódkönyv: $S = \{11001011, 00110110, 11011101, 01100100, 01010110\}$

- **Kódszóhossz (n):** Minden kódszó 8 bites, így $n = 8$.
- **Minimális távolság ($d(S)$):** Számoljuk ki a páronkénti Hamming-távolságokat:
 - $d(11001011, 00110110) = 7$
 - $d(11001011, 11011101) = 3$
 - $d(11001011, 01100100) = 6$
 - $d(11001011, 01010110) = 5$
 - $d(00110110, 11011101) = 6$
 - $d(00110110, 01100100) = 3$
 - $d(00110110, 01010110) = 2$ (Ez a legkisebb érték)
 - $d(11011101, 01100100) = 5$
 - $d(11011101, 01010110) = 4$
 - $d(01100100, 01010110) = 3$
 - Eredmény: $d(S) = 2$.
- **Képességek:**
 - Hibafelismerés: $d - 1 = 2 - 1 = 1$ hiba ismerhető fel.
 - Hibajavítás: $\lfloor \frac{2-1}{2} \rfloor = 0$ hiba javítható.

CRC (Cyclic Redundancy Check)



A CRC egy ciklikus hibadetektáló kód, amit széles körben használnak hálózatokban és adattárolásnál.

- Küldő oldal: A hasznos üzenethez ($M(x)$) hozzáilleszt r darab nullát (ahol r a $G(x)$ generátor polinóm fokszáma). Elosztja $G(x)$ -szel modulo-2 aritmetikával.
- Az osztás maradéka ($r(x)$) lesz a biztonsági ellenőrzőösszeg.
- Az elküldött keret: $T(x) = x^r \cdot M(x) + r(x)$.
- Vevő oldal: Elosztja a kapott $T(x) + E(x)$ üzenetet $G(x)$ -szel. Ha a maradék 0, nem történt felismerhető hiba. Ha a maradék nem 0, hiba történt az átvitel során.

Feladatok – Kidolgozás és Megoldás

1. Feladat: Paritásbit hozzáadása 7-bites kódszavakhoz

Feladat: A 7 bites kódszavak alkotják a kódkönyvtárat. Adjunk hozzá egy paritásbitet, ami 1 ha a kódszóban lévő egyesek száma páratlan, és 0 ha páros.

- Adj példát!
- Mekkora ezen kód Hamming-távolsága?
- Mennyi egyszerű és milyen hosszú löketszerű hibát képes kezelni?

Megoldás:

- **Példa:**
 - Adat: 1011001 (4 darab 1-es, páros) → paritásbit: 0. Kódolt szó: 10110010.
 - Adat: 1101000 (3 darab 1-es, páratlan) → paritásbit: 1. Kódolt szó: 11010001.
- **Hamming-távolság:** $d(S) = 2$. Mivel minden érvényes kódolt szóban páros számú 1-es van, legalább 2 bitnek kell megváltoznia ahhoz, hogy egy másik érvényes (páros 1-est tartalmazó) kódszót kapjunk.
- **Hibakezelés:**
 - Egyszerű hiba: $d - 1 = 1$ hiba (és minden páratlan számú bit-hiba) garantáltan felismerhető.
 - Hibajavítás: $\lfloor \frac{2-1}{2} \rfloor = 0$ hiba javítható.
 - Löketszerű hiba: Bármilyen páratlan számú bit-hibát tartalmazó löketszerű hiba felismerhető.

2. Feladat: Két paritásbit alkalmazása

Feladat: Olyan hibajelző sémát alkalmazunk, amelyben két paritás-bit van: az egyik a páros, a másik a páratlan pozíciójú bitek ellenőrzésére.

- Adj példát!
- Mekkora ezen kód Hamming-távolsága?
- Mennyi egyszerű és milyen hosszú löketszerű hibát képes kezelni?

Megoldás:

- **Példa:** 6-bites adat: $b_1 b_2 b_3 b_4 b_5 b_6$.
 - p_1 a páratlan pozíciók (b_1, b_3, b_5) paritása, p_2 a páros pozíciók (b_2, b_4, b_6) paritása.
 - Adat: 101100. Páratlan bitek (1, 1, 0) összege páros → $p_1 = 0$. Páros bitek (0, 1, 0) összege páratlan → $p_2 = 1$.
 - Kódolt üzenet: 10110001.
- **Hamming-távolság:** $d(S) = 2$. Egyetlen bit változása az adatban pontosan az egyik paritásbitet megváltoztatja, így az érvénytelen lesz.
- **Hibakezelés:**
 - Egyszerű hiba felismerése: $d - 1 = 1$ bit hiba biztosan felismerhető.
 - Löketszerű hiba: Egy legfeljebb 2 bit hosszú löketszerű hiba (burst) egy páros és egy páratlan pozíciójú bitet érint, így mindkét paritásbit jelezni fogja a hibát. Ezért 2 bit hosszúságú löketszerű hiba biztosan felismerhető.

3. Feladat: Mátrixos paritási technika ($k \times l$ mátrix)

Feladat: Az n küldendő adatbitet egy $k \times l$ bit-mátrixnak tekintjük. Minden oszlophoz kiszámolunk egy paritásbitet, amellyel kiegészítjük a mátrixot egy új sorban. Az adatokat soronként küldjük el.

- Hogy viselkedik ez a módszer egyszerű és löketszerű bit-hibák esetén?
- Milyen hosszú lehet egy bitsorozat, melynek minden bitje hibás, hogy a hibát felismerjük?
- Egészítsd ki a mátrixot egy új oszloppal is (két dimenziós paritás). Hogyan használható ez 1-bithiba javítására?
- Mi a helyzet több bithibával és löketszerű hibákkal?

Megoldás:

- **Működés:** Mivel az átvitel soronként történik, egy legfeljebb l (oszlopok száma) hosszúságú löketszerű hiba a mátrixban legfeljebb 1 bitet ront el minden oszlopban.
- **Felismerhető hiba hossza:** Bármely legfeljebb l bit hosszú csupa hibás bitsorozat 100%-os biztonsággal felismerhető, mert minden oszlopban pontosan 1 bit-hiba keletkezik, amit az adott oszlop paritásbitje kimutat.
- **2D paritás (soros és oszlopos paritás) 1-bithiba javítására:**
 - Ha a mátrixot egy sor-paritás oszloppal is kiegészítjük, pontosan 1 bit elromlásakor pontosan 1 sor-paritás és 1 oszlop-paritás fog elromlani.
 - A hibás sor és oszlop metszéspontjában lévő bit az elromlott bit, így az a bit invertálásával javítható!
- **Több bithiba:** 2 hiba felismerhető, de nem mindig javítható. Ha 4 hiba téglalap alakot alkot (2 azonos sor és 2 azonos oszlop metszéspontja), a paritások nem változnak, így a hiba rejtve marad.

4. Feladat: CRC kiszámítása

Feladat: Számolja ki a CRC ellenőrzőösszeget az 1110.1101.0011.0100.0111 üzenetre, ha a generátor polinóm $G(x) = x^4 + x^2 + x + 1$!

- Adja meg a biztonsági bitsort, amit a küldő kiegészít!
- Mutasson példát olyan bithibára, amit nem képes felismerni a fenti módszer!

Megoldás:

- **Adatok:** Üzenet $M = 11101101001101000111$ (20 bit). Generátor polinóm $G(x) = x^4 + x^2 + x + 1$, bitmintája 10111 ($r = 4$).
- **Lépések:**
 - Mátrix/bitek kiegészítése $r = 4$ darab 0-val: 111011010011010001110000.
 - Modulo-2 polinomosztás elvégzése 10111-gyel:
 - Az osztás maradéka: 0101.
- **Biztonsági bitsor (CRC ellenőrzőösszeg):** 0101.
- **Elküldött keret:** 111011010011010001110101.
- **Észrevétlen hiba példa:** Bármilyen hiba-vektor $E(x)$, amely megegyezik a generátor polinómmal, azaz $E(x) = G(x) = 10111$. Ha az üzenet első 5 bitjében történik ez a hiba, a vevőnél elvégzett osztás maradéka továbbra is 0 lesz, így a hiba rejtve marad.

5. Feladat: CRC ellenőrzés a vevő oldalon

Feladat: Történt-e hiba az átvétel során, ha a fogadó a következő üzenetet kapja: 1101.0011.0000.1010 és a generátor polinóm $G(x) = x^4 + x^2 + 1$?

Megoldás:

- **Adatok:** Fogadott üzenet: 1101001100001010 (16 bit). Generátor polinóm: $G(x) = x^4 + x^2 + 1$, bitmintája 10101 ($r = 4$).
- **Ellenőrzés:** Elosztjuk a teljes fogadott üzenetet modulo-2 aritmetikával a 10101 bitmintával.
- **Osztás maradéka:** 1011.
- **Következtetés:** Mivel a kapott maradék nem nullasorozat (1011 $\neq$ 0000), **átviteli hiba történhetett** az üzenetben!

Wireshark Hálózati Elemző

Mi a Wireshark?

A Wireshark egy ingyenes, nyílt forráskódú hálózati forgalomelemző és csomagelfogó alkalmazás (érhető el Linux, Windows és macOS rendszerekre).

- **Fő funkciók:** Csomagok valós idejű elfogása, megjelenítése és részletes szűrése.
- **Fontos korlát:** NEM tűzfal, nem tudja a forgalmat korlátozni vagy blokkolni, csak megfigyelni!

Használati lépések

- **Felvétel készítése:** Interfész kiválasztása → indítás → forgalom rögzítése → leállítás.
- **PCAP betöltése:** Korábban elmentett .pcap / .pcapng fájlok megnyitása elemzésre.
- **SSLlog beállítása:** TLS/SSL titkosított forgalom dekódolása előre elmentett premaster secret kulcsfájl megadásával.

Felület szerkezete

1. **Szűrési sáv:** A megjelenítendő csomagok szűrésére szolgál.
2. **Csomaglista:** A rögzített csomagok időrendi sorszámmal, forrás/cél IP-címmel és protokollal.
3. **Csomag részletei (rétegek):** Az kiválasztott csomag OSI/TCP-IP rétegeinek részletes lebontása (Ethernet, IP, TCP/UDP, alkalmazási réteg).

Szűrési szintaxis és operátorok

- **Protokoll alapú szűrés:** pl. http, dns, arp, icmp.
- **Mező alapú szűrés:** pl. tcp.flags.ack vagy tcp.dstport == 80.
- **Logikai operátorok:** and, or, not, xor.

Parancssori Hálózati Eszközök

1. **ipconfig / ifconfig**

Megmutatja a számítógép hálózati interfészeinek IP-címét, alhálózati maszkját és az alapértelmezett átjárót.

- Windows példa: `ipconfig`
- Linux/macOS példa: `ifconfig` vagy `ip addr`

2. **nslookup**

DNS lekérdezések végrehajtására szolgál. Megadja, hogy egy domain névhez milyen IP-cím tartozik (vagy fordítva).

- **Gyakori rekord típusok:**
 - A: IPv4 cím
 - AAAA: IPv6 cím
 - MX: Mail server (levelezőszerver)
 - NS: Name server (névszerver)
- **Példa:** `nslookup www.google.com`

3. **ping**

Ellenőrzi, hogy egy távoli host elérhető-e a hálózaton, és méri az oda-vissza válaszidőt (RTT).

- ICMP Echo Request csomagokat küld a célállomásra, és méri a visszatérő ICMP Echo Reply válaszidejét.
- **Példák:**
 - `ping www.google.com`
 - Windows (5 csomag): `ping -n 5 8.8.8.8`
 - Linux/macOS (5 csomag): `ping -c 5 8.8.8.8`

4. **tracert / traceroute**

Feltérképezi és megmutatja a hálózati útvonalat (routereket), amelyen keresztül a csomag eljut a célállomáshoz.

- Működése: Fokozatosan növeli az IP-csomagok TTL (Time-To-Live) értékét (1, 2, 3...), így az útvonalon lévő minden router ICMP "Time Exceeded" üzenetet küld vissza a saját címével.
- **Példa:** `tracert www.google.com` (Linuxon: `traceroute www.google.com`)

A HTTP Protokoll Fejlődése (HTTP/1 - HTTP/3)

HTTP/1.0 (1996)

- **Alapja:** TCP protokollra épül.
- **Működése:** Minden egyes HTTP kéréshez külön TCP kapcsolatot nyit a kliens a szerver felé, majd a válasz után lezárja azt.
- **Hátránya:** Rendkívül lassú és nagy overhead, mivel minden erőforrás letöltésénél le kell futnia a TCP 3-utas kézfogásnak (3-way handshake).

HTTP/1.1 (1997)

- **Keep-Alive (Persistent Connections):** Egyetlen TCP kapcsolat újrafelhasználható több egymást követő HTTP kéréshez és válaszhoz, csökkentve a kézfogási késleltetést.
- **HTTP Pipelining:** A kliens elküldhet több kérést anélkül, hogy megvárna az előzőekre adott válaszokat. A válaszoknak szigorúan a kérések sorrendjében kell megérkezniük.
 - **Probléma (Head-of-Line Blocking):** Ha egy korábbi kérés feldolgozása elakad, az az összes mögötte lévő kérés válaszát blokkolja az adott kapcsolaton.
 - **Gyakorlati megoldás:** A böngészők párhuzamosan több (általában max 6) különálló TCP kapcsolatot tartanak fenn ugyanahhoz a szerverhez.

HTTP/2.0 (2015)

- **Multiplexelés (HTTP Streams):** Egyetlen TCP kapcsolaton belül több független kérés-válasz adatfolyam (stream) továbbítható párhuzamosan, sorrendiség kötöttsége nélkül.
- **Eredmény:** Megszünteti a Head-of-Line (HOL) blocking problémát az alkalmazási rétegben.
- **Server Push:** A szerver képes kezdeményezni erőforrások küldését a kliens felé még azelőtt, hogy a kliens kifejezetten kérné azokat.
- **Megmaradt korlát (Transport Layer HOL Blocking):** Mivel a szállítási rétegben továbbra is egyetlen TCP kapcsolatot használ, ha 1 TCP csomag elvész a hálózaton, a TCP torlódáskezelése és újraküldése miatt az összes stream blokkolódik a kapcsolat helyreállásáig.

HTTP/3.0 (2020 / 2022)

- **QUIC Protokoll:** A HTTP/3 már nem TCP-t, hanem a UDP-re épülő új QUIC szállítási protokollt használja.
- **Szállítási rétegbeli stream-ek:** A stream-ek a szállítási réteg első osztályú állampolgárai. Ha egy stream-ben csomagvesztés történik, az kizárólag az adott stream-et érinti, a többi független stream zavartalanul halad tovább.
- **Sima hálózati váltás (Connection ID):** A QUIC nem az IP-cím és port négyesével, hanem egy egyedi Connection ID segítségével azonosítja a kapcsolatot. Ez lehetővé teszi, hogy pl. okostelefonon Wi-Fi-ről 4G/5G-re váltva a hálózati kapcsolat megszakadás nélkül folytonos maradjon.